DUOC UC — ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Desarrollo Fullstack II (DSY1104) — Evaluación Parcial 2

Pastelería Mil Sabores

Especificación de Requisitos de Software (ERS)

Versión 2.0 (Auditado contra Código Fuente)

Documento:	Especificación de Requisitos de Software (ERS V2)
Asignatura:	Desarrollo Fullstack II (DSY1104)
Institución:	Duoc UC
Integrantes:	*[Nombre del Alumno 1 / Nombre del Alumno 2]*
Docente:	*[Nombre del Profesor / Docente]*
Fecha de Auditoría:	17 de Octubre de 2026
Estado:	Aprobado / Auditado contra Código Fuente

1. Control de Versiones

El presente historial registra la evolución técnica y metodológica del proyecto entre las dos primeras etapas evaluativas:

Versión	Fecha	Autores	Descripción del Cambio y Alcance Técnico
1.0	Evaluación Parcial 1	*[Equipo de Desarrollo EP1]*	Propuesta inicial de arquitectura, catálogo estático básico (3 productos), maquetación estática HTML/CSS y definición conceptual del sistema de pastelería artesanal.
2.0	17 de Octubre de 2026	*[Equipo de Desarrollo EP2]*	Refactorización completa a SPA con React 19 y Vite 6. Implementación de Atomic Design, React Router con 16 rutas reales, catálogo ampliado a 28 productos con fotografías locales, carrito con cálculo de totales y precios efectivos en oferta, checkout con validación estricta, confirmación de compra y compra fallida, panel de administración interactivo con operaciones CRUD y persistencia en LocalStorage (claves milSabores_*). Suite de 25 archivos de pruebas automatizadas con Vitest alcanzando 89.54% de cobertura de líneas.

2. Introducción

2.1 Propósito del Documento

El presente documento formal de Especificación de Requisitos de Software (ERS V2) define con precisión los requisitos funcionales y no funcionales, la arquitectura de software, el modelo de datos, la suite de pruebas unitarias y las reglas de negocio de la aplicación **Pastelería Mil Sabores**. Ha sido auditado contra el código fuente real del proyecto para la Evaluación Parcial 2 de la asignatura Desarrollo Fullstack II (DSY1104) de Duoc UC.

2.2 Alcance del Sistema

Pastelería Mil Sabores es una Single Page Application (SPA) para comercio electrónico de repostería artesanal tradicional chilena e internacional. Permite navegar por un catálogo con 28 productos categorizados, buscar productos por texto, consultar fichas técnicas con fotografías locales de alta definición, gestionar un carrito interactivo, tramitar pedidos con captura de datos de despacho y persistir compras en el almacenamiento local del navegador (LocalStorage). Incluye además un panel administrativo para gestión de inventario y artículos de blog gastronómico.

2.3 Objetivos del Proyecto

- Desarrollar una interfaz moderna y 100% responsiva implementando la metodología Atomic Design y utilitarios de Bootstrap 5.
- Desplegar un catálogo gastronómico amplio con 28 productos variados en 8 categorías, respaldado por fotografías locales de alta resolución.
- Proveer una experiencia de compra interactiva: catálogo, detalle, carrito con recálculo dinámico de precios de oferta, checkout estructurado y pantallas de compra exitosa y fallida.
- Separar de forma estricta las vistas de la lógica de negocio mediante una capa de servicios modulares (`productosService`, `usuariosService`, `carritoService`, `pedidosService` y `blogService`).
- Alcanzar altos estándares de calidad con 171 pruebas unitarias en Vitest con cobertura de código superior al 88%.

2.4 Definiciones y Abreviaturas

Término / Acrónimo	Definición en el Contexto del Proyecto
SPA	Single Page Application: Aplicación web donde toda la navegación e interactividad se ejecuta en el navegador del cliente mediante JavaScript sin recargas de página completas.
Atomic Design	Metodología de diseño de interfaces que jerarquiza componentes en Átomos, Moléculas, Organismos y Plantillas/Páginas.
LocalStorage	Mecanismo de la Web Storage API para persistir datos como texto serializado (JSON) en el navegador del usuario de forma indefinida.
CRUD	Acrónimo de Create, Read, Update, Delete (Crear, Leer, Actualizar y Eliminar), correspondiente a la gestión de productos en el panel administrativo.
Vitest	Framework de pruebas unitarias de alto rendimiento nativo para Vite, compatible con Jest y Testing Library.

3. Descripción General

3.1 Contexto del Negocio

Pastelería Mil Sabores es un emprendimiento enfocado en la elaboración y venta de repostería artesanal tradicional chilena (alfajores de maicena, brazo de reina, empolvados, torta de milhojas) y pastelería fina contemporánea (cheesecakes, tartas, brownies, cupcakes). La versión 2 digitaliza el catálogo con fotografías locales de alta calidad y automatiza el proceso de cotización y solicitud de pedidos.

3.2 Actores del Sistema

Actor	Descripción y Permisos
Visitante Anónimo	Cualquier usuario de la web. Puede ver el Home, Catálogo (28 productos), Categorías, Detalle de Producto, Ofertas, Nosotros, Blog y Contacto. Puede agregar productos a su carrito y registrarse o iniciar sesión.
Cliente Registrado	Usuario autenticado con rol "cliente". Posee acceso a Checkout para ingresar datos de entrega y confirmar la orden de compra.
Administrador	Usuario con rol "admin" (cuenta predefinida `admin@milsabores.cl`). Posee acceso exclusivo a la ruta protegida `/admin` para administrar el catálogo (listar, crear, editar, eliminar productos).

3.3 Entorno de Ejecución y Dependencias Reales

- **React:** 19.0.0 (Librería UI declarativa basada en componentes funcionales y hooks)
- **React DOM:** 19.0.0
- **React Router DOM:** 7.18.4 (Enrutador de cliente para SPA)
- **Bootstrap:** 5.3.8 (Estilos responsivos, grilla flexbox y componentes visuales)
- **Vite:** 6.3.1 (Empaquetador y servidor de desarrollo ultrarrápido)
- **Vitest:** 5.0.5 y **@vitest/coverage-v8:** 5.0.5 (Framework de pruebas automatizadas)
- **Testing Library:** @testing-library/react 16.3.3 y @testing-library/jest-dom 7.0.1

3.4 Restricciones, Supuestos y Limitaciones

> **Restricciones de Alcance Académico:**
> 1. La aplicación no posee un servidor backend ni base de datos relacional centralizada; toda la persistencia opera en el LocalStorage del cliente.
> 2. Los métodos de pago en Checkout son simulaciones visuales que no ejecutan cobros bancarios reales.
> 3. Los formularios de contacto y checkout validan y simulan el envío sin comunicarse con un servidor SMTP.
> 4. Las contraseñas y sesiones se gestionan en LocalStorage sin firma criptográfica JWT de backend, lo cual es una limitación reconocida del alcance frontend.

4. Requisitos Funcionales

A continuación se detallan los 18 Requisitos Funcionales auditados y confirmados en el código fuente:

RF-01: Navegación Global y Página de Inicio (Home)

Actor: Visitante / Cliente / Admin | **Ruta:** `/` (Home.jsx), Navbar, Footer

El sistema debe presentar una página principal con barra de navegación institucional, banner héroe con fotografía representativa, sección de categorías rápidas, vitrina de productos destacados y sección dedicada a la repostería tradicional chilena.

> **Criterio de Aceptación:** La barra de navegación permite acceder a todas las secciones; el banner y los postres destacados se visualizan responsivamente sin errores de renderizado.

RF-02: Catálogo General con Fotografías Locales de Alta Resolución

Actor: Visitante / Cliente / Admin | **Ruta:** `/productos` (Productos.jsx), ProductCard.jsx

El sistema debe presentar un catálogo de 28 productos gastronómicos organizados en tarjetas responsivas con fotografía real local (/images/productos/*), nombre, categoría, precio en pesos chilenos (CLP) y botones de acción.

> **Criterio de Aceptación:** El catálogo carga los 28 productos con sus respectivas fotografías desde la carpeta pública y permite paginar o explorar fluidamente.

RF-03: Búsqueda Predictiva y Filtrado por Categorías

Actor: Visitante / Cliente | **Rutas:** `/productos` y `/categorias`, SearchBar.jsx

El usuario debe poder filtrar los productos por categoría gastronómica (Tortas, Pasteles, Galletas, Cupcakes, Panadería, Tartas, Especiales, Sin Azúcar) y buscar en tiempo real por texto libre dentro de nombre y descripción.

> **Criterio de Aceptación:** Al escribir en la barra de búsqueda o activar un botón de categoría, la lista de productos se actualiza inmediatamente sin recargar la página.

RF-04: Detalle Exhaustivo del Producto

Actor: Visitante / Cliente | **Ruta:** `/productos/:id` (DetalleProducto.jsx)

El sistema debe mostrar la ficha técnica completa del producto seleccionado: fotografía ampliada, nombre, descripción detallada, categoría, precio normal, precio de oferta (si aplica), selector de cantidad y botón para añadir al carrito.

> **Criterio de Aceptación:** Si el ID ingresado en la URL no existe en el catálogo, la página muestra un mensaje amigable indicando que el producto no fue encontrado y ofrece un enlace de retorno al catálogo.

RF-05: Sección de Ofertas y Precios Efectivos de Venta

Actor: Visitante / Cliente | **Ruta:** `/ofertas` (Ofertas.jsx), getPrecioEfectivo()

El sistema debe listar exclusivamente aquellos productos que tengan la propiedad enOferta: true y un precioOferta válido. La función getPrecioEfectivo() debe garantizar que se aplique el precio rebajado en todas las vistas.

> **Criterio de Aceptación:** La página muestra la insignia de oferta, el precio normal tachado, el precio de oferta destacado y los cálculos en el carrito concuerdan de forma idéntica.

RF-06: Gestión Interactiva del Carrito de Compras

Actor: Visitante / Cliente | **Ruta:** `/carrito` (Carrito.jsx), CartItem.jsx, CartSummary.jsx, carritoService.js

El usuario debe poder visualizar los ítems agregados, modificar cantidades (incrementar/decrementar con enteros >= 1), eliminar productos, vaciar el carrito y observar el cálculo automático del subtotal y total en CLP.

> **Criterio de Aceptación:** Los cambios de cantidad recalculan el total al instante y persisten en la clave milSabores_carrito de LocalStorage.

RF-07: Proceso de Checkout y Validación de Despacho

Actor: Cliente Registrado | **Ruta:** `/checkout` (Checkout.jsx), pedidosService.js

El sistema debe exigir un formulario completo de entrega (nombre, email con formato válido, teléfono de 8 a 15 dígitos, dirección, comuna, región e indicaciones opcionales). Si el usuario no ha iniciado sesión, es redirigido a /login.

> **Criterio de Aceptación:** Al completar correctamente todos los campos y confirmar, se genera el pedido, se vacía el carrito y se redirige a /compra-exitosa. Si ocurre un fallo en almacenamiento, se redirige a /compra-fallida.

RF-08: Comprobante de Compra Exitosa y Notificación de Compra Fallida

Actor: Cliente Registrado | **Rutas:** `/compra-exitosa` (CompraExitosa.jsx) y `/compra-fallida` (CompraFallida.jsx)

El sistema debe mostrar el comprobante formal con código único de pedido (prefijo PED-timestamp-random), fecha formateada, desglose de ítems, total y datos del cliente. En caso de error, CompraFallida explica el problema y ofrece reintentar sin perder los ítems del carrito.

> **Criterio de Aceptación:** El pedido confirmado puede ser recuperado por su ID o consultando el último pedido registrado en milSabores_ultimo_pedido.

RF-09: Registro de Clientes Nuevos

Actor: Visitante | **Ruta:** `/registro` (Registro.jsx), usuariosService.js

El sistema debe permitir a nuevos usuarios crear una cuenta ingresando nombre, correo electrónico y contraseña, impidiendo el registro de correos duplicados.

> **Criterio de Aceptación:** Al registrarse con éxito, el usuario es almacenado en la clave milSabores_usuarios de LocalStorage y se le redirige al inicio de sesión.

RF-10: Autenticación e Inicio de Sesión de Usuarios

Actor: Visitante / Cliente / Admin | **Ruta:** `/login` (Login.jsx), usuariosService.js

El usuario debe poder autenticarse con correo y contraseña. Si las credenciales son válidas, se genera la sesión en la clave milSabores_sesion; si el rol es admin redirige a /admin, y si es cliente redirige al catálogo o checkout.

> **Criterio de Aceptación:** Credenciales erróneas despliegan una alerta de error en pantalla sin reiniciar el formulario ni recargar el navegador.

RF-11: Cierre de Sesión (Logout)

Actor: Cliente / Administrador | **Componente:** Navbar.jsx, usuariosService.js

El usuario con sesión activa debe poder cerrar su sesión haciendo clic en la opción Cerrar sesión del menú superior.

> **Criterio de Aceptación:** La sesión es removida de LocalStorage (milSabores_sesion) y la interfaz se actualiza a estado visitante no autenticado.

RF-12: Panel de Administración y Operaciones CRUD de Productos

Actor: Administrador | **Ruta:** `/admin` (Admin.jsx), productosService.js

El administrador debe poder gestionar el catálogo en una vista unificada: buscar productos, dar de alta un producto nuevo, editar cualquier producto existente (nombre, precio, categoría, imagen, oferta, precioOferta) y eliminar productos previa confirmación modal de confirmación.

> **Criterio de Aceptación:** Las operaciones de creación, edición y eliminación actualizan en tiempo real la tabla de inventario y persisten en milSabores_productos.

RF-13: Protección de Rutas Administrativas (RutaProtegida)

Actor: Todos los Actores | **Componente:** RutaProtegida.jsx (organisms)

El acceso a /admin debe estar estrictamente condicionado. Si un visitante sin sesión intenta ingresar, es redirigido a /login. Si un usuario con rol cliente intenta ingresar, es redirigido a la raíz /.

> **Criterio de Aceptación:** El usuario sin privilegios administrativos no puede visualizar ni interactuar con la interfaz de gestión.

RF-14: Página Institucional "Nosotros"

Actor: Visitante / Cliente | **Ruta:** `/nosotros` (Nosotros.jsx)

Presenta la historia de la pastelería, su propuesta de valor artesanal, pilares de calidad e higiene, y fotografías reales de la cocina y la tienda.

> **Criterio de Aceptación:** La página se adapta fluidamente a cualquier pantalla e incorpora testimonios y datos de contacto del taller.

RF-15: Blog Gastronómico y Artículos Editoriales

Actor: Visitante / Cliente | **Rutas:** `/blog` (Blog.jsx) y `/blog/:id` (DetalleBlog.jsx), blogService.js

El sistema ofrece un módulo editorial con artículos especializados sobre técnicas de repostería chilena y consejos de horneado, permitiendo navegar entre el listado general y la lectura completa de cada publicación.

> **Criterio de Aceptación:** Los artículos se renderizan con fecha, categoría, autor y contenido formateado con imágenes de cabecera.

RF-16: Formulario de Contacto y Atención al Cliente

Actor: Visitante / Cliente | **Ruta:** `/contacto` (Contacto.jsx)

Provee un formulario para enviar consultas o solicitudes especiales (nombre, correo, asunto y mensaje) con validación completa y confirmación visual de envío.

> **Criterio de Aceptación:** Campos vacíos o correos inválidos muestran alertas contextuales; al enviar con datos correctos se muestra un mensaje de éxito sin recargar la página.

RF-17: Persistencia y Migración Automática de Catálogo (Versión 3)

Actor: Sistema | **Servicio:** productosService.js (CATALOG_VERSION = 3)

El sistema compara la versión almacenada en milSabores_catalogVersion con la constante CATALOG_VERSION (3). Si la versión es obsoleta o no existe, migra automáticamente e incorpora los 28 productos base preservando los productos adicionales creados por el administrador.

> **Criterio de Aceptación:** Al cargar la aplicación por primera vez o limpiar la caché, los 28 productos y sus fotografías se inicializan automáticamente sin errores.

RF-18: Tolerancia a Imágenes Faltantes (Fallback Visual)

Actor: Sistema | **Componentes:** ProductCard.jsx, DetalleProducto.jsx, Admin.jsx

Los componentes que renderizan imágenes de productos incorporan el manejador onError para reemplazar automáticamente cualquier imagen no disponible por la imagen de respaldo /images/productos/default-pasteleria.jpg.

> **Criterio de Aceptación:** Ninguna tarjeta o ficha técnica muestra iconos de imagen rota en el navegador.

5. Requisitos No Funcionales

ID	Categoría	Descripción y Criterio de Verificación Auditado
RNF-01	Responsividad	La interfaz debe adaptarse fluidamente a dispositivos móviles (360px+), tablets (768px+) y pantallas de escritorio (1200px+) utilizando el sistema de grillas flexbox de Bootstrap 5.3.8.
RNF-02	Usabilidad y Accesibilidad	Formularios con etiquetas semánticas (`label` , `aria-label` , `role="alert"`), contrastes adecuados, alertas de error contextuales y retroalimentación inmediata ante acciones del usuario.
RNF-03	Rendimiento y Optimización	Las 28 fotografías de productos y recursos visuales se sirven localmente desde `public/images/productos/` con atributo `loading="lazy"` en tarjetas para acelerar el renderizado inicial.
RNF-04	Tolerancia a Fallos de Recursos	Manejador de error (`onError`) en imágenes que garantiza la carga automática de `default-pasteleria.jpg` si una ruta gráfica fallase.
RNF-05	Arquitectura Modular (Atomic Design)	Estructuración estricta de componentes en `src/components/` : átomos (Alert, Badge, Button, Input), moléculas (FormField, QuantityControl, SearchBar), organismos (CartItem, CartSummary, Footer, Navbar, ProductCard, RutaProtegida) y plantillas (AuthLayout, MainLayout, PageHeader).
RNF-06	Calidad de Código y Testing	Suite de pruebas unitarias automatizadas en Vitest con 25 archivos de especificación (`.spec.jsx`) y 171 pruebas, alcanzando un **89.54% de cobertura de líneas** (superando la meta institucional del 80%).
RNF-07	Compatibilidad de Entornos	Funcionamiento verificado en los principales navegadores modernos: Google Chrome, Microsoft Edge, Mozilla Firefox y Safari, sin dependencias propietarias no estándar.
RNF-08	Seguridad en Cliente y Limitaciones	Validación preventiva de datos y sanitización con `trim()` en formularios. Nota académica: Al carecer de backend en esta etapa, las credenciales y pedidos se persisten en LocalStorage sin cifrado asimétrico.

6. Arquitectura y Diseño

6.1 Estilo Arquitectónico: SPA con React 19 y Vite 6

La aplicación adopta el patrón Single Page Application (SPA), donde toda la interfaz se renderiza de forma reactiva en el cliente. La navegación declarativa se gestiona mediante React Router DOM (v7.18.4) envuelta en la plantilla estructural `MainLayout.jsx` .

6.2 Componentes Organizados con Atomic Design

La estructura de componentes en `src/components/` sigue estrictamente la jerarquía de diseño atómico:

- **Átomos (src/components/atoms/):** Componentes elementales reutilizables: `Alert.jsx`, `Badge.jsx`, `Button.jsx`, `Input.jsx`.
- **Moléculas (src/components/molecules/):** Combinaciones de átomos con propósito funcional: `FormField.jsx` (input con label y mensaje de error), `QuantityControl.jsx` (botones +/- con contador) y `SearchBar.jsx` (input con botón de limpieza).
- **Organismos (src/components/organisms/):** Secciones complejas de interfaz: `CartItem.jsx`, `CartSummary.jsx`, `Footer.jsx`, `Navbar.jsx`, `ProductCard.jsx` y `RutaProtegida.jsx`.
- **Plantillas (src/components/templates/):** Estructuras de diseño generales: `AuthLayout.jsx`, `MainLayout.jsx` (navbar + contenedor principal + footer) y `PageHeader.jsx` (cabecera con título, subtítulo y descripción).

6.3 Separación de Responsabilidades y Patrones de Diseño

1. **Service Layer Pattern:** La lógica de datos y persistencia se encapsula de forma aislada en `src/services/` (`productosService.js`, `usuariosService.js`, `carritoService.js`, `pedidosService.js` y `blogService.js`). Las páginas no realizan manipulación directa de LocalStorage ni reglas de negocio complejas.
2. **Container / Presenter Pattern:** Los componentes visuales (como ProductCard, CartItem, Alert) actúan como componentes presentacionales que reciben datos y callbacks por props, mientras que las páginas (Productos, Carrito, Admin) gestionan el estado y la invocación de servicios.
3. **Module Pattern (ES Modules):** Cada servicio expone únicamente las funciones necesarias (`export function ...`) manteniendo privadas las claves de almacenamiento y funciones auxiliares.
4. **Route Guard Pattern:** El componente `RutaProtegida` actúa como guardián de rutas, interceptando el acceso a `/admin` y redirigiendo según el estado de la sesión.

7. Modelo de Datos y Persistencia Local

Las entidades de datos se almacenan en LocalStorage como cadenas serializadas en formato JSON. A continuación se detallan las claves y estructuras de datos reales verificadas en los servicios:

Clave en LocalStorage	Servicio Responsable	Estructura de Datos y Campos Clave
`milSabores_productos`	`productosService.js`	Array de objetos: `id` (number), `nombre` (string), `categoria` (string), `precio` (number), `descripcion` (string), `imagen` (string), `enOferta` (boolean), `precioOferta` (number, opcional).
`milSabores_catalogVersion`	`productosService.js`	Valor string numérico (`"3"`) que valida la sincronización automática del catálogo base con los 28 productos.
`milSabores_usuarios`	`usuariosService.js`	Array de usuarios: `id` (number: Date.now()), `nombre` (string), `email` (string en minúsculas), `password` (string), `rol` ("cliente" | "admin").
`milSabores_sesion`	`usuariosService.js`	Objeto de sesión activa: `id` (number), `nombre` (string), `email` (string), `rol` ("cliente" | "admin"). Se remueve al cerrar sesión.
`milSabores_carrito`	`carritoService.js`	Array de ítems: `id` (number), `nombre` (string), `precio` (number: precio efectivo), `imagen` (string), `cantidad` (number entero >= 1).
`milSabores_pedidos`	`pedidosService.js`	Array de órdenes: `id` ("PED-timestamp-random"), `fecha` (ISO string), `items` ([{ id, nombre, precio, cantidad, subtotal }]), `total` (number), `cantidadTotal` (number), `cliente` ({ nombre, email, telefono, direccion, comuna, region, indicaciones }), `estado` ("confirmado").
`milSabores_ultimo_pedido`	`pedidosService.js`	Objeto de la última orden confirmada, utilizado por CompraExitosa.jsx como respaldo en caso de recarga de página.

8. Mapa de Rutas y Navegación del Sistema

La aplicación cuenta exactamente con **16 rutas declaradas en** `src/App.jsx`. La siguiente tabla presenta el inventario completo y verificado de rutas:

N°	Ruta URL	Página / Componente	Nivel de Acceso	Propósito y Funcionalidad
1	`/`	`Home.jsx`	Público	Página principal, banner héroe, categorías rápidas y repostería chilena.
2	`/productos`	`Productos.jsx`	Público	Catálogo de 28 productos con búsqueda y filtros por categoría.
3	`/productos/:id`	`DetalleProducto.jsx`	Público	Ficha técnica completa, ingredientes, fotos y selector de cantidad.
4	`/ofertas`	`Ofertas.jsx`	Público	Listado exclusivo de productos con precio de oferta activo.
5	`/categorias`	`Categorias.jsx`	Público	Directorio interactivo de las 8 categorías gastronómicas.
6	`/nosotros`	`Nosotros.jsx`	Público	Historia, pilares institucionales y fotografías de la cocina.
7	`/blog`	`Blog.jsx`	Público	Artículos formativos sobre técnicas de repostería.
8	`/blog/:id`	`DetalleBlog.jsx`	Público	Lectura completa de un artículo editorial de blog.
9	`/contacto`	`Contacto.jsx`	Público	Formulario de contacto, atención al cliente y ubicación.
10	`/login`	`Login.jsx`	Público	Formulario de inicio de sesión para clientes y administrador.
11	`/registro`	`Registro.jsx`	Público	Formulario de creación de cuenta para nuevos clientes.
12	`/carrito`	`Carrito.jsx`	Público	Resumen del pedido, modificación de cantidades y totales.
13	`/checkout`	`Checkout.jsx`	Privado (Cliente)	Captura de datos de despacho y confirmación de la orden.
14	`/compra-exitosa`	`CompraExitosa.jsx`	Público / Cliente	Pantalla de confirmación con código PED- y detalle de orden.
15	`/compra-fallida`	`CompraFallida.jsx`	Público / Cliente	Notificación de error en pedido y opciones de reintento.
16	`/admin`	`Admin.jsx`	Privado (Admin)	Panel administrativo unificado con operaciones CRUD

N°	Ruta URL	Página / Componente	Nivel de Acceso	Propósito y Funcionalidad
				completas.

9. Reglas de Negocio y Validaciones

- **Regla de Precios Efectivos (`getPrecioEfectivo`):** Si un producto posee `enOferta === true` y su `precioOferta` es un número positivo menor al precio normal, el precio de venta en catálogo, detalle y carrito es exactamente `precioOferta`. Si no, rige el precio regular.
- **Regla de Control de Cantidades en Carrito:** La cantidad de un producto en el carrito debe ser un número entero mayor o igual a 1 (`Number.isInteger(cantidad) && cantidad >= 1`). No se permiten cantidades negativas ni fraccionadas.
- **Regla de Validación de Despacho en Checkout:** Todo pedido requiere: Nombre no vacío, Email con formato RFC 5322 (`/^[^s@]+@[^s@]+.[^s@]+$/`), Teléfono numérico válido de entre 8 y 15 dígitos, Dirección, Comuna y Región no vacías.
- **Regla de Generación de ID de Pedido:** Los pedidos confirmados generan un identificador único con estructura `PED-${Date.now()}-${random4Digitos}` .
- **Regla de Control de Acceso (`RutaProtegida`):** El acceso a `/admin` exige sesión activa con rol admin. Si no hay usuario se redirige a `/login` ; si el rol es cliente se redirige a `/` .
- **Regla de Creación de Productos en Admin:** Los productos nuevos creados por el administrador deben tener nombre no vacío, descripción no vacía, categoría válida, precio entero positivo y, si se activa oferta, `precioOferta` entero positivo estrictamente menor al precio normal.

10. Pruebas Unitarias y Cobertura de Código

10.1 Entorno de Testing y Configuración

El proyecto cuenta con un entorno de pruebas automatizadas configurado en `vite.config.js` mediante **Vitest (v5.0.5)** con entorno `jsdom` . Las pruebas se ubican en el directorio `test/` con archivos de especificación `.spec.jsx` y un archivo central de configuración `test/setup.js` que proporciona mocks limpios de LocalStorage y eventos del navegador.

10.2 Comando de Ejecución Oficial

```
npm run test:coverage   (o en Windows PowerShell: npm.cmd run test:coverage)
```

10.3 Resultados Reales Obtenidos en la Ejecución Oficial

> **Auditoría Automatizada Realizada:** Ejecutada y auditada el **17 de Octubre de 2026** a las **16:38:36**.
> • **Total de Archivos de Prueba:** 25 pasados (25 total).
> • **Total de Pruebas Unitarias:** 171 pasadas (171 total, 0 fallos).
> • **Tiempo de Ejecución:** 24.86 segundos.

Métrica de Cobertura (v8)	Porcentaje Real Obtenido	Umbral Mínimo Requerido	Estado de Aprobación
Líneas (% Lines)	**89.54%**	80.00%	**Aprobado Excedido**
Ramas (% Branch)	**90.35%**	80.00%	**Aprobado Excedido**
Funciones (% Funcs)	**89.23%**	80.00%	**Aprobado Excedido**
Sentencias (% Stmts)	**88.95%**	80.00%	**Aprobado Excedido**

10.4 Cobertura por Capas y Módulos

- `src/App.jsx`: 100% líneas, 100% funciones.
- `src/components/atoms` (Alert, Badge, Button, Input): 100% líneas, 100% funciones.
- `src/components/molecules` (FormField, QuantityControl, SearchBar): 100% líneas, 100% funciones.
- `src/components/organisms` (CartItem, CartSummary, Footer, Navbar, ProductCard, RutaProtegida): 88.57% líneas.
- `src/components/templates` (AuthLayout, MainLayout, PageHeader): 100% líneas, 100% funciones.
- `src/services` (blogService, carritoService, pedidosService, productosService, usuariosService): 91.02% líneas.

11. Matriz de Trazabilidad de Requisitos

Requisito	Página / Vista	Componentes / Servicios Involucrados	Archivo de Prueba (test/*.spec.jsx)
RF-01	Home.jsx	Navbar.jsx, Footer.jsx, ProductCard.jsx	Home.spec.jsx, Navbar.spec.jsx
RF-02	Productos.jsx	ProductCard.jsx, productosService.js	Productos.spec.jsx, ProductCard.spec.jsx
RF-03	Productos.jsx, Categorias.jsx	SearchBar.jsx, productosService.js	Categorias.spec.jsx, Productos.spec.jsx
RF-04	DetalleProducto.jsx	QuantityControl.jsx, Button.jsx, Badge.jsx	DetalleProducto.spec.jsx
RF-05	Ofertas.jsx	ProductCard.jsx, getPrecioEfectivo()	Ofertas.spec.jsx, productosService.spec.jsx
RF-06	Carrito.jsx	CartItem.jsx, CartSummary.jsx, carritoService.js	Carrito.spec.jsx, carritoService.spec.jsx
RF-07	Checkout.jsx	FormField.jsx, Input.jsx, pedidosService.js	Checkout.spec.jsx, pedidosService.spec.jsx
RF-08	CompraExitosa.jsx, CompraFallida.jsx	pedidosService.js, Alert.jsx	CompraExitosa.spec.jsx, CompraFallida.spec.jsx
RF-09	Registro.jsx	FormField.jsx, usuariosService.js	Registro.spec.jsx, usuariosService.spec.jsx
RF-10	Login.jsx	FormField.jsx, usuariosService.js	Login.spec.jsx, usuariosService.spec.jsx
RF-12	Admin.jsx	productosService.js, FormField.jsx	Admin.spec.jsx, productosService.spec.jsx
RF-13	Ruta /admin	RutaProtegida.jsx (organisms)	RutaProtegida.spec.jsx
RF-14/15/16	Nosotros.jsx, Blog.jsx, Contacto.jsx	blogService.js, PageHeader.jsx, Alert.jsx	Nosotros.spec.jsx, Blog.spec.jsx, Contacto.spec.jsx

12. Limitaciones y Trabajo Futuro

12.1 Limitaciones Declaradas de la Versión 2

- **Persistencia Local Exclusiva:** Los datos residen en el navegador individual del usuario (LocalStorage). No existe sincronización en la nube entre diferentes navegadores o computadores.

- **Cobro Bancario Simulado:** El proceso de checkout no se conecta con Webpay Plus o pasarelas de pago reales.
- **Mensajería de Correo Simulada:** El sistema no envía correos de confirmación ni notificaciones por SMTP.
- **Seguridad Básica en Frontend:** Al carecer de servidor de autenticación con JWT y cookies HttpOnly, las contraseñas se almacenan en texto plano dentro de LocalStorage, acorde al alcance frontend de la evaluación.

12.2 Trabajo Futuro (Propuesta para Evaluación Parcial 3)

- Implementación de un backend centralizado con Node.js, Express y base de datos relacional PostgreSQL.
- Autenticación robusta con JSON Web Tokens (JWT) y cifrado de contraseñas con bcrypt.
- Integración con la pasarela de pago Transbank Webpay Plus o Mercado Pago SDK.
- Integración de servicio SMTP automatizado para emisión de comprobantes de compra por correo electrónico.

13. Conclusión

La versión 2 de **Pastelería Mil Sabores** representa una modernización completa y rigurosa del frontend. Se ha consolidado una Single Page Application robusta, elegante y modular fundamentada en Atomic Design, React 19, Vite 6 y Bootstrap 5.

Con un catálogo ampliado a 28 preparaciones con fotografías reales locales, 16 rutas operativas en React Router, carrito interactivo con cálculo estricto de precios efectivos, checkout validado, panel administrativo CRUD y una suite de 25 archivos de pruebas automatizadas con Vitest que alcanza un **89.54% de cobertura de líneas**, el proyecto cumple con los más altos estándares técnicos y metodológicos exigidos para la Evaluación Parcial 2 de Desarrollo Fullstack II (DSY1104) en Duoc UC.